EP-L2: Elastic Payload Storage for In-Place Writeback in GPU L2 Caches

Author information to be completed
English working draft EN-v4 — not for submission

***Abstract*—In a fixed-partition GPU L2, a dirty victim may need to move from resident storage to a writeback buffer even though only its consumer has changed. The copy consumes bank service, while static role quotas prevent idle capacity from being borrowed. Elastic Payload L2 (EP-L2) separates resident-tag visibility, physical data ownership, and writeback ordering. A dirty victim becomes writeback-staged in its original slot; the incoming resident receives a distinct, bank-compatible slot. The design retains the reference's payload capacity and bank-service budget, eliminating resident-to-writeback copying without adding resident tags or miss-tracking entries. We specify the ownership protocol and a matched evaluation that separates copy elimination, relief of staging-capacity pressure, and application performance. Independently qualified application measurements and physical implementation results remain pending; this working draft makes no empirical speedup, area, or energy claim.**

***Index Terms*—GPU, L2 cache, writeback, tag–data decoupling, resource sharing**

I. Introduction

A cache line's visibility, data, and transaction state need not have the same lifetime. On a dirty eviction, the old resident tag can become reusable before the lower memory system accepts the data. Same-address ordering may persist even after that transfer. A design must preserve the data and ordering obligations without unnecessarily retaining the original tag or moving data merely to change their role.

We study an L2 with fixed quotas for resident, bypass-return, and writeback-buffer (WBUF) payloads. Its banked reference copies a dirty victim into WBUF before reusing the resident slot. The copy requires a source read and destination write; sending the staged line requires another read. A larger WBUF can relieve a quota limit but does not remove the copy. Conversely, fewer copy operations need not improve execution time if another resource determines progress.

The design insight is to transfer ownership rather than location. In EP-L2, the dirty payload remains in place, while an independently allocated slot receives the incoming resident line. Resident and WB allocations can borrow free slots across the fixed role partitions, while resident tags stay fixed. This WB-only path does not establish concurrent dynamic bypass service. The mechanism can thus reduce internal movement and expose idle slots to transient consumers; these are distinct potential benefits.

Tag–data indirection and storage unification have been explored previously. V-Way decouples tag and data placement [1]; unified GPU local memory reallocates capacity among registers, cache, and scratchpad [2]; in-cache MSHR storage and MiCache reuse cache storage for pending-request information [3], [4]. Our focus is instead the *resident-to-writeback data lifetime*: retaining dirty data in place while separately releasing tag visibility and address-ordering state. This transition requires evaluating both data-service demand and access to otherwise idle capacity.

This paper presents the in-place writeback protocol and its bounded resource model, then organizes evaluation around three questions: which bank operations disappear, whether static staging capacity actually blocks progress, and whether useful L2 service improves application execution. Optional victim-retention and read-only paths are scoped separately; they are not prerequisites for the primary writeback comparison.

II. EP-L2 Design

A. One-Slice Organization and Matched Reference

Figure 1 places the payload pool beside the distinct lookup, request-tracking, and ordering structures. The primary reference, B16 (`B16_BANKED`), has 1024 resident, 128 bypass-return, and 16 WBUF payload slots. U-WB16 (`U_WB16`) dynamically shares the same 1168 slots. Each slot is 128 B, with four 32-B sectors. Both modes provision four banks of 292 slots and at most one line-sized operation per bank per *physical* GPU cycle. Both also provision a common 128-B copy latch outside the pool and 128 WAD/control entries. Sixteen is the reference's WB data quota, not a new U-WB control limit.

A tag names a payload by {slot identifier, generation}. Each slot has one authoritative owner; valid/pending sector masks and pins protect its consumers. The initial design retains the resident bank class,

$$\text{bank}(p) = p \bmod 4 = \text{tag_index} \bmod 4. \tag{1}$$

A globally free slot is therefore not necessarily a legal landing slot for a particular replacement. The fixed 1024 resident tags remain the lookup-capacity limit, even when resident data use previously nonresident slots.

B. Reservation, In-Place Transition, and Release

Before replacing a dirty line, the controller reserves the WAD entry, required issue resources, and a legal data destination. In B16, the old data are read from the resident bank and written through the latch to WBUF. Only after that copy is safe can the original resident slot be reused. WBUF subsequently supplies a bank read for the lower send.

In U-WB16, a dirty line A retains payload p and changes role to `WB_STAGED`. A different slot q is reserved for the new resident B; its tag points to q, not p. This removes

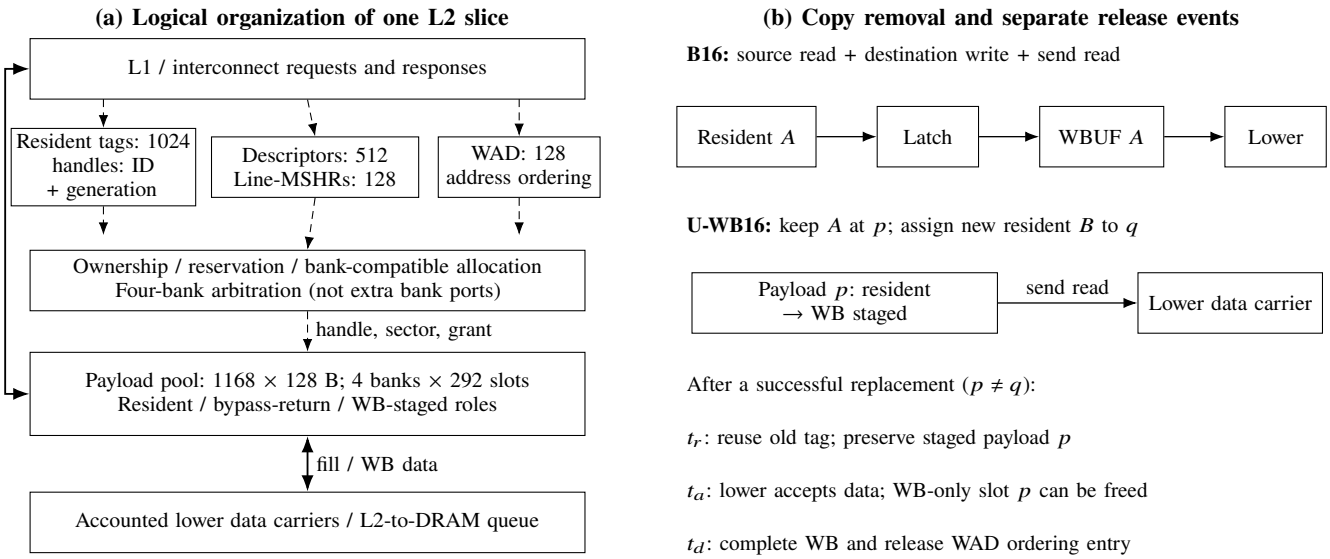


Fig. 1. Declared WB-only design, not a physical floorplan or a measured cycle schedule. Dashed links denote control dependencies; solid links denote data movement. B16 uses fixed 1024/128/16 role quotas over the same banks; U-WB16 shares those slots. Both retain a 128-B copy latch outside the pool. Optional TVD/RO consumers are not drawn as extra data arrays.

resident-to-WBUF copying, but retains the send read, lower-queue occupancy, and receiving data carrier. Pre-reservation refusals leave visible ownership unchanged; bank retries can retain reservations. A failed final commit restores ownership and releases reservations, without erasing bank work already performed. Accepted requests always have a reserved return location.

At committed replacement time t_r, the tag can be reused; the staged payload survives until actual lower data acceptance t_a. The writeback address directory (WAD) maintains ordering until real completion t_d. Merely enqueueing a software pointer does not establish that the lower path owns the data. The receiving carrier must be accounted for before the local staging slot is released. A late event for A uses its transaction/handle identity and cannot modify B through a reused tag index.

For a tracked read, its requester descriptor and payload pin survive until the response is accepted by the L2-to-interconnect queue (or an internal response is consumed). The Line-MSHR persists until its last tracked requester retires; payload pending-sector state remains separately checked. Neither retirement event releases the independent WB WAD. Partial readiness and stale generations cannot authorize payload reuse. WB draining needs no new resident miss; eventual bank/lower service is assumed, not whole-GPU deadlock freedom.

C. Optional Consumers and Implementation Cost

TVD and RO are separate, uncompiled source candidates, not qualified production extensions. TVD is specified to retain victim payloads without resident tags, using a shared WAD/TVD budget of 128 address entries and one additional lookup cycle after a tag miss. Valid-sector reads can be served in place; clean writes/atomics require promotion, while dirty writes/atomics wait for writeback completion. Retained data remain explicitly owned. The RO extension separately controls pending-tag detachment and Line-MSHR bypass under a complete-commandlist, line-granular read-only oracle. Detached fills complete original consumers without reinstalling replaced tags. Both extensions are OFF in the primary B16/U-WB16 experiment and receive no extra payload array or descriptor pool.

Primary payload storage is 146 KiB/slice, or 9.125 MiB over 64 slices; common latches add 8 KiB/chip. An identifier needs $P = \lceil \log_2 1168 \rceil = 11$ bits. Explicit tag handles cost $1024(P + G)$ bits for generation width G; up to 128 WB handles add $128(P + G)$ bits plus state and packet binding. Shared baseline fields are not counted again as pooling overhead. Generation width, pin bounds, allocation selection, arbitration, rollback state, and lookup ports require a concrete hardware mapping. Handle validation includes generation/owner comparisons; bank-compatible free-slot selection and bounded oldest-ready arbitration are not free hardware. Symbolic verification versions are not payload RAM or automatically hardware fields. Equal storage and bank budgets do not establish equal area or frequency; matched RTL, SRAM macros, and timing/power measurements remain necessary.

III. Evaluation Methodology

A. Controlled Comparison and Workloads

The implementation uses Accel-Sim's SASS flow and the GPGPU-Sim performance-model lineage [5], [6]. These citations identify the tools; their hardware-correlation results do not validate our modified L2. The same independently qualified repaired binary must execute both modes with identical complete traces, non-mode configurations, timing scope, and termination rules. Table I lists the declared research configuration, not undocumented commercial-GPU internals.

The development set is convolutionSeparable, FWT_7_21, cfd_097k, vectorAdd_4M, and sad, the predeclared negative

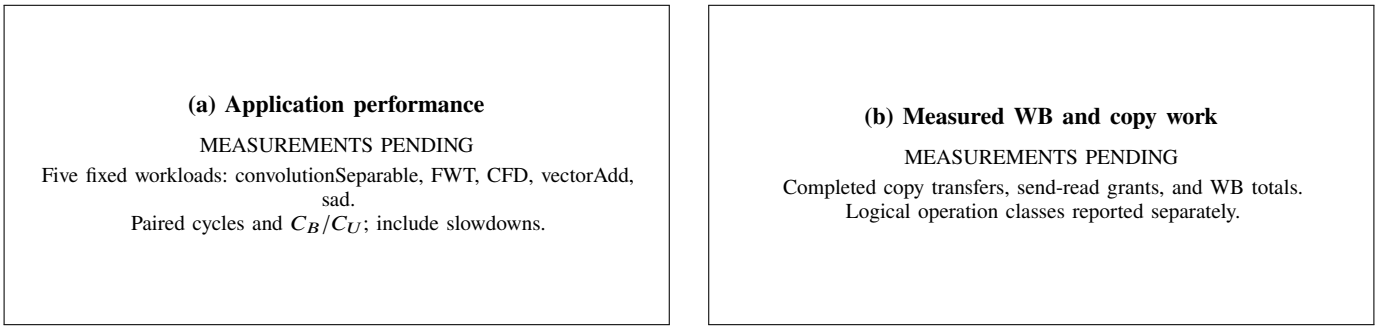


Fig. 2. Reserved main-result area. The two panels will use the same current-qualified binary and matched inputs. No bars, values, reference line, or mean have been fabricated; blank space is intentional.

TABLE I
MATCHED EVALUATION CONFIGURATION (PER SLICE UNLESS STATED).

Resource	Setting
Resident / total payload	1024 tags / 1168 slots (146 KiB)
Line / sectors / banks	128 B / 4×32 B / 4×292 slots
Bank service / copy latch	1 op/bank/physical cycle / 128 B
Descriptors / Line-MSHRs	512 / 128; at most 32 requesters/address
WAD / L2-to-DRAM queue	128 / 128 entries
Chip / memory channels	64 slices / 32 channels
DRAM / scheduler / return	850 MHz / 128 / 192 entries per channel
L1 (fixed)	64 KiB/core; 4 banks; 20-cycle setting
WBUF8 sensitivity	1160 slots, 145 KiB; 4×290 bankslots

control whose pressure remains to be checked. Btree and dwt2d are reserved for validation after policy selection, with scan scheduled separately. Slowdowns remain in the result set. A five-workload geometric mean is computed only after every pair satisfies the common criteria, rather than averaging a favorable completed subset.

B. Correctness and Timing Qualification

Qualification requires directed tests on actual cache/partition paths, not only detached helper models. The repaired candidate has not yet run these production tests. Required coverage includes nonzero host-memcpy offsets, at most one bank grant per physical cycle, valid host-prefill payload identity, generation reuse, sector returns, response backpressure, and distinct WB data-acceptance/completion events. Native tag-ordering timestamps must not alter bank-service time. Host prefill rejects dirty, pinned, pending, or nonquiescent state before mutation.

Complete numerical payloads are unavailable in the SASS traces. The supported checks are therefore ownership, symbolic sector versions, requester/transaction identity, and final resource drain, not full numerical application correctness. Source/binary/configuration/trace identities and current independent acceptance accompany every result. Normal exit or a historical certificate cannot override later qualification withdrawal.

IV. Results and Causal Analysis

Result status. No currently accepted application pair is available for this revision. Figures 2–4 are fixed-size reservations for actual measurements, not plotted results. Earlier invalidated runs are excluded. The following specifies how each panel will support or reject a claim; findings will replace this text after acceptance.

A. End-to-End Performance and Copy Work

Figure 2(a) is reserved for the full five-workload B16/U-WB16 comparison. With matching instruction counts and timing scope, speedup is $S = C_{B16}/C_{U-WB16}$. Pairwise cycles accompany the figure, and negative controls and regressions remain visible.

For one committed, non-rolled-back WB episode, the specified reference performs three logical staging/send bank operations, whereas the in-place path performs the send read only. This is a protocol count, not a 3× speedup. Actual runs may evict different numbers of lines; Fig. 2(b) therefore reports measured copy/send operations and WB totals in both modes. Retry attempts are not new logical operations; work consumed before rollback still counts. Per-episode reduction cannot replace application totals. The native operation classes group copy reads with send reads and copy writes with resident writes; their physical split requires an audited event mapping.

B. Capacity Pressure, Service, and Downstream Effects

A nonzero borrowing counter does not demonstrate a capacity benefit. Figure 3(a) relates WB-data-full waits to staged/reserved occupancy, pool-full or bank-compatible landing failures, and separately recorded WAD pressure. A capacity claim needs binding static quota pressure *and* useful allocation from otherwise inaccessible slots. An idle bypass partition may supply headroom in this workload set, but does not establish the best static partition.

Figure 3(b) separates logical operations, physical bank grants, and waiting. Summed per-operation waits are not elapsed GPU cycles and cannot be directly subtracted from application time. Fig. 3(c) tests whether improved admission merely transfers pressure to the lower queue, scheduler, or DRAM. WB-data release at lower acceptance and WAD release at completion must remain distinguishable. Together, these panels connect copy/capacity changes to useful service rather than only to counters that happen to decrease.

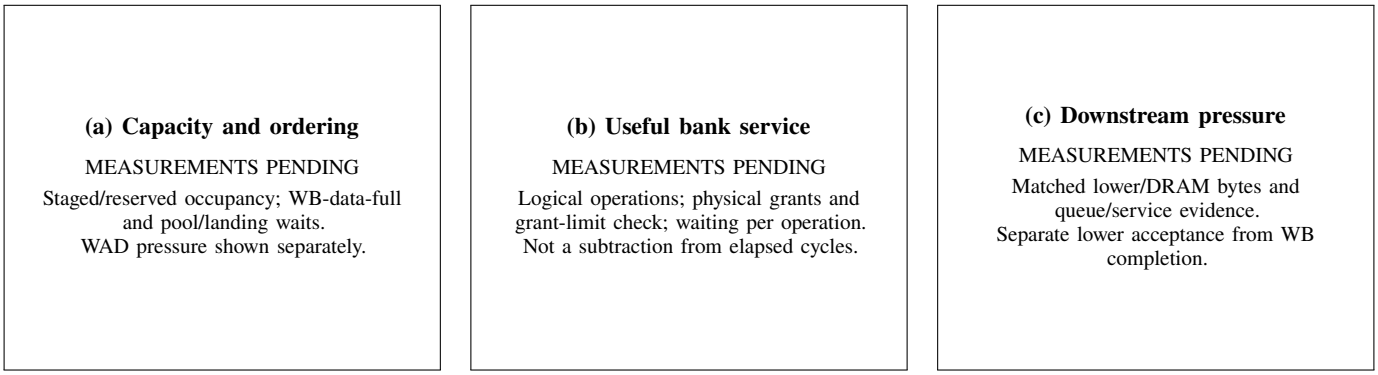


Fig. 3. Reserved causal-analysis area. These three panels distinguish staging capacity, bank service, and pressure transferred downstream. Metrics with different units must not share an unlabeled scale. The future data determine the conclusion, not the panel titles.

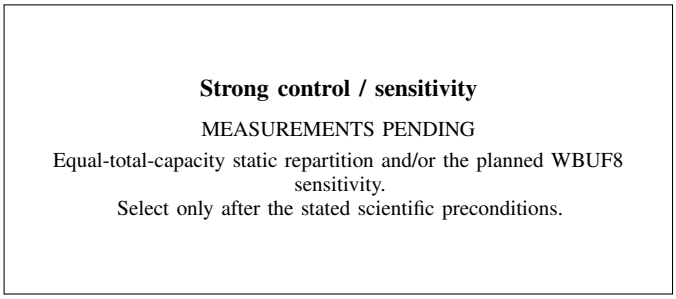


Fig. 4. Reserved control panel. A fixed repartition tests whether an allocation chosen in advance suffices; a copy-service diagnostic tests a different cause. This slot is not a claim that either experiment has run.

C. Strong Controls and Cost Sensitivity

A capacity-sharing claim requires a strong equal-1168-slot static control, such as 1024 resident/112 bypass/32 WBUF, charging its reduced bypass quota. This repartition is not executable in the frozen 8/16-entry interface; it needs an explicit implementation and independent qualification. An independently qualified equal-copy-service diagnostic may be needed when copy elimination could explain the difference. The planned, still unqualified WBUF8 sensitivity checks a second budget; these comparisons do not add free metadata or bank ports.

If local service improves without end-to-end benefit, a representative case crosses baseline/mechanism with original/one justified downstream-headroom setting. This four-point interaction test differs from simply showing that faster memory helps both modes. Ideal WB-capacity, metadata, and read-hit diagnostics require a separate same-binary baseline and remain optimistic diagnostics, not implemented EP-L2 performance. Optional TVD/RO results enter only after their own qualification; the reserved panels do not authorize new experiments.

The final interpretation must also include added control latency and implementation cost. Equal data bytes are not equal area, and fewer bank accesses alone do not establish energy savings. Macro energy, arbitration/selection, wiring, leakage, and execution duration need compatible implementation inputs. No physical PPA number is available in this revision.

V. Related Work and Scope

Flexible storage organization. V-Way increases tag capacity relative to data and uses indirection for demand-based associativity [1]. Tag-Split Cache instead changes data-storage granularity while retaining coarse request granularity [7]. EP-L2 fixes resident tags and line/sector sizes, targeting cross-role payload lifetimes rather than either form of spatial packing.

Storage reused for transient state. In-cache MSHR designs reuse cache storage for pending-load information, subject to placement and port constraints [3]. MiCache shares FPGA storage between cache lines and MSHR subentries, converting MSHR entries to cache entries on response [4]. EP-L2 instead retains existing dirty *data* during resident-to-WB transition, with request metadata and WAD separately bounded.

Other resource and access policies. Gebhart *et al.* repartition SM-local registers/cache/scratchpad storage at kernel boundaries; their evaluated cache is write-through [2]. DL-MSHR links slots to adapt primary/secondary-miss capacity; its slots can also carry write-miss data [8]. MRPB reorders and bypasses requests before GPU L1 access [9], whereas victim caching retains displaced lines for reuse [10]. These address related resource limits but are not direct implementations of the WB-only comparison. Potential combinations require measurement rather than assumed additive benefit.

VI. Conclusion

EP-L2 transfers dirty-payload ownership without relocating the data, while separating resident visibility and WB ordering. Its architectural effects are fewer copy operations and more flexible access to a fixed payload budget. Their workload value must be established through matched performance, true capacity pressure, service, and cost evidence. The reserved result panels make that remaining evaluation explicit without presenting unmeasured benefits.

References

[1] M. K. Qureshi, D. Thompson, and Y. N. Patt, "The V-Way cache: Demand-based associativity via global replacement," in *Proceedings of the 32nd Annual International Symposium on Computer Architecture (ISCA)*, 2005, pp. 544–555.

[2] M. Gebhart, S. W. Keckler, B. Khailany, R. Krashinsky, and W. J. Dally, "Unifying primary cache, scratch, and register file memories in a throughput processor," in *Proceedings of the 45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2012, pp. 96–106.

[3] K. I. Farkas and N. P. Jouppi, "Complexity/performance tradeoffs with non-blocking loads," Digital Equipment Corporation, Western Research Laboratory, Research Report 94/3, Mar. 1994.

[4] S. Xu, S. Lu, Z. Shao, X. Liao, and H. Jin, "MiCache: An MSHR-inclusive non-blocking cache design for FPGAs," in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA)*, 2024, pp. 22–32.

[5] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-Sim: An extensible simulation framework for validated GPU modeling," in *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 473–486.

[6] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, "Analyzing CUDA workloads using a detailed GPU simulator," in *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2009, pp. 163–174.

[7] L. Li, A. B. Hayes, S. L. Song, and E. Z. Zhang, "Tag-split cache for efficient GPGPU cache utilization," in *Proceedings of the 2016 International Conference on Supercomputing (ICS)*, 2016, pp. 43:1–43:12.

[8] Y. Gu and L. Chen, "Dynamically linked MSHRs for adaptive miss handling in GPUs," in *Proceedings of the International Conference on Supercomputing (ICS)*, 2019, pp. 510–521.

[9] W. Jia, K. A. Shaw, and M. Martonosi, "MRPB: Memory request prioritization for massively parallel processors," in *Proceedings of the 20th IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2014, pp. 272–283.

[10] N. P. Jouppi, "Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," in *Proceedings of the 17th Annual International Symposium on Computer Architecture (ISCA)*, 1990, pp. 364–373.